

Web 6

Thursday, 2 April 2026 00:41

这套系统到底是什么

它不是“终端包一层模型 API”的小工具，而是一个完整 Agent 宿主。UI、状态、工具、任务和扩展面都被统一收进同一个 runtime。

入口

`main.tsx`

内核

`QueryEngine` + `query`

扩展

MCP / Plugin / Skill

1. 交互宿主层

`main.tsx`、`interactiveHelpers.tsx`、`screens/REPL.tsx`

负责进程入口、setup flow、Ink 渲染、prompt 输入、消息列表、权限弹窗、任务面板。

↓

2. 状态与运行时层

`bootstrap/state.ts` + `state/AppStateStore.ts`

前者管全局 session/runtime 元数据，后者管 UI 和当前会话可变状态。

↓

3. 会话内核层

`QueryEngine.ts` + `query.ts`

管理 conversation 生命周期、query loop、tool_use 回注、compact、budget、fallback。

↓

4. 执行抽象层

`commands.ts`、`tools.ts`、`Task.ts`、`tasks/*`

统一命令、工具和后台任务的运行模型。

↓

5. 扩展与外部服务层

`services/api`、`services/mcp`、`utils/plugins`、`skills`

把模型、MCP、插件、技能和网络能力接入宿主。

一句话：这是一个以终端 REPL 为壳、以 Query loop 为内核、以 Tool / Command / Task 为执行抽象的完整 Agent Runtime。

小红书

rednote ID: 102720456

用户一次输入后，系统怎么跑

真正的中心不是“聊天框”，而是 query 主循环。输入进来后，系统会先组装上下文，再向模型流式请求，再把工具调用结果回注给模型，直到本轮完成。

- A. 启动与装配**
main.tsx 预处理 profile, keychain, MDM, env, config; init.ts 初始化网络、遥测、安全环境。cleanup, scratchpad.
- B. 进入 REPL / SDK 宿主**
交互式走 screens/REPL.tsx，非交互式走 headless / SDK 路径，但都共享同一套会话内核。
- C. submitMessage**
QueryEngine.submitMessage() 接收 prompt，持有 messages、usage、abort controller、file cache。
- D. query loop**
query.ts 组装 system prompt / user context，发起流式 API 请求，持续接收 assistant 内容和 tool_use。
- E. 工具执行**
toolOrchestration.ts / StreamingToolExecutor.ts 按并发性执行工具，再把 tool_result 回注给模型继续跑。
- F. 完成本轮**
更新 usage、session storage、telemetry、AppState 和 UI，必要时进入 compact / budget continuation / fallback 分支。

模块	在主链路中的职责
QueryEngine.ts	conversation 生命周期容器
query.ts	单轮 query 主循环与状态迁移
Tool.ts	统一工具上下文、权限、进度、AppState 接口
services/api/claude.ts	模型 API 流式请求与消息规范化
services/tools/*	工具并发控制、执行、取消、错误传播

关键点：这套系统真正复杂的地方不是 prompt，而是“query loop + tool runtime + permission model + state sync”这一整条执行链。

评论

rednote ID: 102720456

为什么 Claude Code 是产品级 Agent 架构

从扩展方式和工程抽象看，这个仓库已经明显是产品化形态。它不是不断加旁路，而是把新能力收敛到已有的 runtime 原语里。

Tool Runtime 做了统一治理

- `Tool.ts` 把工具统一成 schema、permission context、AppState、notification、abort、progress 接口。
- `tools.ts` 是工具注册中心，不同 feature/env 下动态裁剪。
- `bash`、文件、web、MCP 等都不是散落调用，而是统一 tool protocol。

Task 是一等公民

- `Task.ts` 定义 task type/status/context/id。
- `tasks/*` 实现 local shell、local agent、remote agent、workflow、dream 等。
- 后台能力可追踪、可取消、可前后台切换、可映射到 UI panel。

扩展面被统一收敛

- MCP: `services/mcp/client.ts` 管 transport、OAuth、resource、tool call。
- Plugin: `pluginLoader.ts` 管发现、校验、缓存、manifest、commands、hooks、agents。
- Skill: `loadSkillsDir.ts` 从 project/user/managed/plugin/bundled/MCP 多源加载 markdown skill。

设计判断	意味着什么
双层状态系统	runtime state 和 UI state 被显式分离，能同时支撑 REPL 与 SDK/headless
feature-gate 很重	代码可按构建、用户、实验配置做裁剪，不是一次性 demo
启动阶段大量 lazy/预取	明显在优化首屏延迟，说明已经是产品工程思维
扩展都映射到已有原语	新增能力不会不断制造新旁路，架构可持续演化

最终结论：`claude-code-source` 的价值不只是“能调用工具”，而是它展示了一个 coding agent / terminal agent 如何被做成真正可运行、可扩展、可维护的产品级宿主。

建议阅读顺序: `main.tsx` → `entrypoints/init.ts` → `screens/REPL.tsx` → `QueryEngine.ts` → `query.ts` → `Tool.ts` / `tools.ts` → `Task.ts` → `services/mcp/client.ts` → `utils/plugins/pluginLoader.ts` → `skills/loadSkillsDir.ts`。



rednote ID: 102720456

今天 AI 圈最大的乐子，应该就是 Claude Code 源码泄露。

Claude Code 因为一个相当低级的问题，整套源码直接泄露了。某种意义上，这大概又是 Anthropic 一次“以一己之力推动全球 AI 圈进步”的名场面。

毕竟对很多做 Agent、做 Coding Runtime、做工具链的人来说，平时最难看到的不是模型能力，而是工业级产品到底是怎么落实现的。这次，算是直接把参考答案摊在桌面上了。

源码地址：

<https://github.com/instructkr/claude-code>

翻下来会发现，Claude Code 本身并不是一个普通聊天界面，而是一个运行在终端里的 agentic assistant。它有自己的 core loop，有 tools，有 subagents，有

REDACTED

rednote ID: 102720456

hooks，也有独立的 CLI / IDE 宿主。Anthropic 后续开放的 Agent SDK 也直接说明，SDK 复用了 Claude Code 的 tools、agent loop 和 context management。

这件事很关键。

它意味着 Claude Code 不是给模型包了一层 UI，而是先实现了一套 runtime，再把产品长在 runtime 之上。

所以我这次重点看的，不是“它具体支持哪些功能”，而是它为什么能以一种工业级产品的方式稳定跑起来。

如果只从实现上看，这套系统其实可以拆成四个问题：

它如何承载交互，如何推进回合，如何治理状态，如何约束执行。

1. 最外层不是聊天框，而是一个

REACT

rednote ID: 102720456

有状态的交互宿主

Claude Code 官方文档反复强调，它可以存在于 terminal、IDE、desktop、browser 这些不同宿主里。这说明它的第一层抽象不是“消息组件”，而是“宿主环境”。

宿主要负责的，不只是输入输出，而是整场会话的可见性：

- * 当前任务在做什么
- * 工具是否正在执行
- * 是否需要审批
- * 会话是否可恢复
- * 当前模式是否变化

所以 Claude Code 的 UI 不是一个简单的“回复渲染器”，而是 runtime 的前端壳。它必须持续反映内部状态迁移，而不是

REACT

rednote ID: 102720456

只展示最终答案。

只要系统支持工具执行、任务切换、权限确认、子代理委派，宿主层就天然要承担状态投影的职责。用户操作的也不是一次性对话，而是一个持续运行中的智能会话环境。

2. 核心不是生成文本，而是推进一个回合

Claude Code 官方对核心循环的概括是：gather context、take action、verify results。

这个描述已经足够说明实现方向：系统的基本单位不是“回答”，而是“回合推进”。

也就是说，每次用户输入进来，系统不是直接请求模型吐一段文本，而是构造一轮可持续推进的执行过程。

QWERTY

rednote ID: 102720456

这背后一定存在一层 conversation runtime。这层 runtime 至少要维护这些对象：

- * 当前累计消息
- * 当前上下文裁剪 / 压缩结果
- * 当前可用工具与权限边界
- * 当前正在进行的工具调用
- * 当前是否进入 delegation / hook / interrupt 路径
- * 当前是否满足结束条件

真正难的不是模型能不能继续输出，而是系统能不能持续决定：

- * 这一轮应该继续推理，还是切到工具
- * 工具结果以什么粒度进入上下文
- * 中间失败之后是 retry、fallback，还是直接中断

REDA

rednote ID: 102720456

- * 流式输出与工具调用如何交错

- * 会话历史是否需要压缩

所以 Claude Code 的核心，不是 response generation，而是 turn orchestration。

很多人看 agent 产品，第一反应还是“模型回复得聪不聪明”。但一旦进入真实执行环境，真正决定产品上限的，往往是这一层回合编排能力，而不是单次生成质量。

3. 主循环本质上是一个执行状态机

只要系统支持 tool use，主循环就不再是“请求模型 -> 收到文本 -> 渲染结束”这么简单。Claude 返回 structured tool call，执行发生在应用侧，结果再回注给 Claude，继续下一轮。

这意味着 Claude Code 内部的主循环

rednote ID: 102720456

一定承担如下职责：

- * 组装本轮上下文
- * 暴露当前可用工具描述
- * 发起模型调用
- * 解析 stop reason / tool use
- * 调度工具执行
- * 收集结果并注入下一轮消息
- * 判定是否继续循环或终止

所以从实现范式上看，它更接近一个状态机，而不是一个简单 RPC 包装器。

工业级 agent runtime 和 demo 的差别，也主要出在这里。demo 的控制流在模型外几乎是平的；runtime 的控制流则是分叉的，会在推理、工具、压缩、审批、回退之间持续切换。

系统不会机械地把整段文本丢给模型，

REDA

rednote ID: 102720456

而是先整理上下文、组织系统提示、确定工具与权限，再进入一轮交互。真正持续运转的，不是一次推理请求，而是一套执行循环。

4. 工具系统不是能力清单，而是一套统一调用协议

Claude Code 官方文档和 Agent SDK 都强调，tools 是第一层原语，而且这些 tools 被同一套 loop 和 context management 驱动。真正值得看的，不是它内置了多少工具，而是所有能力都被压进了统一的调用模型里：

模型决策，宿主执行，结果回注。

这意味着工具系统的重点不是丰富，而是抽象统一。

从实现层看，统一抽象至少要解决四件事：

QWERTY

rednote ID: 102720456

- * 输入输出 schema

- * 权限校验入口

- * 执行状态上报

- * 结果回注格式

只有这四件事统一了，文件读写、shell、搜索、MCP、外部服务调用，才会在 runtime 里表现为同一种对象：可调度能力。

否则它们就只是散落函数的集合，不能稳定进入 agent loop。

这也是为什么很多 agent demo 看起来“功能很多”，但一旦规模稍微做大就开始失控。问题不是工具不够，而是没有被纳入同一套调度协议。

5. 控制面和执行面是分开的

Claude Code 官方文档里，CLI

REACT

rednote ID: 102720456

reference 、 settings 、 hooks 、 subagents、tools 本来就是分模块暴露的。这个切分背后，对应的是两套不同的接口面。

一套是执行面，给模型用，完成任务。

一套是控制面，给用户用，管理系统。

只要是终端型 agent，用户就一定不只是在给任务，还会直接管理宿主：

- * 切模式
- * 看状态
- * 调整设置
- * 恢复会话
- * 接插件
- * 做诊断

如果把这些操作混进模型工具里，系统职责会迅速混乱。把它们放进独立控制面，



rednote ID: 102720456

边界才会清晰。

这本质上就是典型的 control plane / execution plane 分离。

这个点很容易被低估。很多人做 agent 时，会把“给模型完成任务的能力”和“给用户管理系统的能力”揉在一起，最后 UI、命令、工具、设置全缠成一团。Claude Code 的做法说明，真正稳定的系统，一开始就得把这两层拆开。

6. Subagents 解决的首先是上下文隔离，而不是“多智能体叙事”

Claude Code 对外公开了 subagents 的配置项：description、tools、permission mode、hooks、model 等。这说明 subagent 在实现上不是一次 prompt hack，而是 runtime 级对象。

REDACTED

rednote ID: 102720456

从实现角度看，subagent 的第一价值不是“并发多个 AI”，而是隔离：

- * 隔离上下文窗口
- * 隔离角色职责
- * 隔离工具权限
- * 隔离中间噪音

主会话接收的是子任务结果，而不是子任务完整执行轨迹。这是非常典型的 runtime 设计。

否则任何复杂任务都会迅速污染主上下文，导致后续推理质量退化。所以 subagent 不是为了显得更高级，而是为了让系统在长任务下仍然保持上下文清洁和职责分明。

换句话说，subagent 首先是一个上下文管理工具，其次才是一个“多智能体”故事。

REDA

rednote ID: 102720456

7. Hooks、skills、memory 对应三类不同的注入点

Claude Code 把 hooks 、subagents、settings、skills 分开讲，本身已经说明这些东西在实现上不是一层。

如果从 runtime 角度整理：

- * skills 是知识 / workflow 块的延迟装载

- * hooks 是生命周期节点上的可执行拦截器

- * memory / context management 是会话状态的保留与裁剪机制

这三者看上去都在“给模型额外信息”，但进入系统的时间点和作用方式完全不同。

skills 解决“该加载什么”；hooks 解决“在什么时候执行什么规则”；memory 解决“哪些历史应该继续存在”。

REACT

rednote ID: 102720456

把这三者在实现上分清，是 runtime 成熟度的重要标志。否则系统会很快变成一堆混在一起的 prompt 注入和旁路逻辑。

8. 权限系统不是附属交互，而是调度逻辑的一部分

Claude Code 一旦能读写文件、运行命令、接入外部工具，权限就不可能只是一个 UI 弹窗。它必须进入主循环。

这意味着权限实现至少会影响三处：

- * 工具是否进入可用集
- * 调度时是否允许自动执行
- * 执行结果是否需要被拦截或中断

所以它不是外围保护层，而是 loop 的组成部分。

真正成熟的实现，权限边界不会只挂在 UI 上，而会下沉到 runtime 对 capability

rednote ID: 102720456

的分发与调度逻辑里。换句话说，权限系统不是“执行之后再确认”，而是“执行之前就参与决策”。

这也是终端型 agent 和普通聊天产品最本质的差别之一。只要系统真的能动文件、跑命令、接服务，权限就不再是一个礼貌性的确认框，而是执行架构的一部分。

9. 从实现上概括，Claude Code 更像一个 runtime，而不是一个工具型产品

Anthropic 现在已经把 Claude Code 的底层能力继续抽成 Agent SDK，对外直接说这是 Claude Code as a library。这个信号很明确：Claude Code 的内部结构不是写死在产品里的，而是已经足够稳定，可以作为库暴露。

所以如果只从技术实现上压缩描述，Claude Code 的核心并不是一个会写代码

REDACTED

rednote ID: 102720456

的对话界面，而是一套面向长期任务的 agent runtime：

- * 最外层是交互宿主
- * 内部是会话引擎和调度主循环
- * 下面挂统一工具协议
- * 再往下是子代理隔离、生命周期 hooks、上下文管理和权限控制

用户输入只是触发点。真正持续运行的，是这一整套 runtime。

这次 Claude Code 泄露，真正值得看的不是“它能做什么”，而是“它为什么能以这种方式跑起来”。

如果只把它理解成一个 AI 写代码工具，会低估它。把它理解成一套终端里的智能执行系统，很多实现细节才会真正变得清晰。

REACT

rednote ID: 102720456